

In this lab, you will see some applications of symbol tables and binary search trees. You may use `std::map` and `std::list` from the STL in Parts A and B; Part C uses an AVL tree (`avl.hpp`, provided). The code in this lab uses C++20 (e.g., `map::contains`); this is the default in GCC 16, and with older compilers add the flag `-std=c++20`.

std::map

The `map<K,V>` associative container in C++ stores a collection of *key-value* pairs where the keys are unique and can be accessed in ascending order. It is implemented as red-black tree. It has the following member functions (and more):

Function	Description	Usage example
insert()	Inserts a pair of the form {key,value} into the map. The key must be unique. Ignore if key is already present in the map.	erp.insert({"Ali",12345})
operator []	Returns a reference to the value associated with the given key. If the key is not present, it inserts a pair {key,x} where x is default value of type V and returns a reference to the value x.	cout << erp["Ali"] erp["Ali"]]=12543
find()	Returns an iterator to the element with the given key. If key is not present in the map, it returns end()	if (erp.find("Zia")==erp.end()) cout << "Not found"
contains() ()	Returns true if the map contains the given key (C++20)	if (!erp.contains("Zia")) cout << "Not found"
erase()	Removes the element with the given key	erp.erase("Ali")
size ()	Returns the number of elements in the map	erp. size ()
empty()	Returns true if the map is empty	erp.empty()

In the following example, we use `map<string, int>` to store erp id of different students. The keys are the names of the students and the values are their erp ids.

```

1 #include <iostream>
2 #include <string>
3 #include <map>
4 using std::cout, std::map, std::string;
5 int main() {
6     map<string, int> erp;
7     erp["Ali"] = 12345;    // insert pair ("Ali", 12345)
8     erp["Ali"] = 13901;    // update value of key "Ali" to 13901
9     erp["Beena"] = 12897; // insert pair ("Beena", 12897)
10    erp.insert({"Beena", 14593}); // ignore, key "Beena" is already present
11    erp.insert({"Chand", 13412}); // insert pair ("Chand", 13412)
12    cout << erp["Ali"] << " " << erp["Beena"] << " " << erp["Chand"] << "\n"; // prints 13901
        12897 13412
13
14    if(erp.find("Dua")==erp.end()) cout << "Dua not present\n"; // "Dua" is not present
15    erp.erase("Beena");    // remove "Beena"
16
17    // Iterate using C++17
18    for (auto& [key, value] : erp)
19        std::cout << '[' << key << "] = " << value << "; ";
20    // C++11 alternative:

```

```

21 // for (auto& n : erp) // n is std::pair<const string, int>
22 //     std::cout << n.first << " = " << n.second << "; ";
23 }

```

std::list

The `list<T>` container in C++ is a doubly linked list that allows for efficient insertions and deletions from both ends as well as from the middle of the list. Unlike vectors, iterators to elements in a list remain valid even after insertions or deletions of other elements. It has the following member functions (and more):

Function	Description	Usage example
<code>push_back()</code> , <code>pop_back()</code>	Insert/remove an element at the end of the list.	<code>lst.push_back(10)</code>
<code>push_front()</code> , <code>pop_front()</code>	Insert/remove an element at the beginning of the list.	<code>lst.push_front(20)</code>
<code>begin()</code> , <code>end()</code>	Returns an iterator to the first / one-past-the-last element of the list.	<code>auto it = lst.begin()</code>
<code>erase()</code>	Removes the element at the position pointed to by the given iterator.	<code>lst.erase(it)</code>

Following is a sample usage of the `list<int>` class:

```

1 #include <iostream>
2 #include <list>
3 using std::cout, std::list;
4
5 int main() {
6     list<int> lst;
7     lst.push_back(10);  lst.push_back(20);  lst.push_back(30);
8     lst.push_front(5);  lst.push_front(1);
9     // List now contains: 1, 5, 10, 20, 30
10
11     cout << "List contents: ";
12     for (int x : lst) // iterate using range-based for loop
13         cout << x << " ";
14     cout << "\n";
15
16     auto it = lst.begin(); ++it; ++it; // iterator to 3rd element (10)
17     lst.erase(it); // remove the element at iterator it (i.e., remove 10)
18
19     cout << "After erasing 3rd element: ";
20     for (int x : lst)
21         cout << x << " ";
22     cout << "\n";
23 }

```

Lab Exercises

The support files contain a header with a stub for each class (`Svector.h`, `LRU.h`, `Mstring.h`), an AVL tree (`avl.hpp`), and a test driver for each part (`test-svector.cpp`, `test-lru.cpp`, `test-mstring.cpp`) whose comments give the expected output. Compile with, e.g., `g++ -std=c++20 test-svector.cpp -o test-svector`.

Part A: Sparse vectors

A *sparse vector* is a vector in which most elements are zero. To save space, we store only the nonzero elements along with their indices in a symbol table. The memory usage is proportional to the number of nonzero entries, rather than the total size of the vector.

Implement **Svector** class to represent a sparse vector of **doubles** with the following methods:

Function	Description
<code>set(int i, double x)</code>	Set the i -th entry to x (if $x = 0$, remove the entry if it exists).
<code>get(int i) const</code>	Return the i -th entry (if not present, return 0).
<code>dot(const Svector& that) const</code>	Return the dot product of this vector with that .
<code>norm() const</code>	Return the Euclidean norm $\ x\ = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$.
<code>add(const Svector& that) const</code>	Return the sum of this vector and that .
<code>scale(double alpha)</code>	Multiply this vector by the scalar alpha (scaling by 0 removes every entry).
<code>print() const</code>	Print the non-zero entries of vector in the form $\{(i_1, x_1), (i_2, x_2), \dots\}$

The operations **set** and **get** should run in $O(\log n)$ time in the worst case, where n is the number of nonzero entries in the vector. Computing the dot product of two vectors should take time proportional to the total number of nonzero entries in both vectors.

Following is a sample usage of the **Svector** class:

```

1 #include "Svector.h" // test-svector.cpp
2 int main() {
3     Svector v1, v2;
4
5     v1.set(1, 3.0); v1.set(3, 4.0); v1.set(10, 5.0);
6     std::cout << "v1 = "; v1.print(); // Expected: {(1,3), (3,4), (10,5)}
7
8     v2.set(1, 2.0); v2.set(2, 7.0); v2.set(3, 1.0);
9     std::cout << "v2 = "; v2.print(); // Expected: {(1,2), (2,7), (3,1)}
10
11     std::cout << "v1.get(2) = " << v1.get(2) << "\n"; // Expected: 0
12     std::cout << "v1 . v2 = " << v1.dot(v2) << "\n"; // Expected: 10
13     std::cout << "||v1|| = " << v1.norm() << "\n"; // Expected: 7.07107
14
15     Svector v3 = v1.add(v2); std::cout << "v1 + v2 = "; v3.print(); // Expected: {(1,5),
16                             (2,7), (3,5), (10,5)}
17     v1.scale(2.0); std::cout << "2 * v1 = "; v1.print(); // Expected: {(1,6),
18                             (3,8), (10,10)}
19     // test removing entry by setting to 0
20     v1.set(3, 0.0); std::cout << "After setting v1[3] = 0: "; v1.print(); // Expected:
    {(1,6), (10,10)}
}

```

Part B: LRU cache

An LRU (*least recently used*) cache stores at most *capacity* items. When an item is accessed and the cache is full, the cache evicts the item that was least recently accessed to make room for it.

Design a data structure that supports two operations, **access** and **remove**. The **access** operation inserts an item if it is not already present (evicting the least recently accessed item if the cache is then over capacity), and marks it as the most recently accessed. The **remove** operation deletes and returns the item that was least recently accessed. Both operations should run in $O(\log n)$ time in the worst case, where n is the number of items currently stored in the data structure.

Hint: Maintain the items in order of access using a doubly linked list. Use a symbol table with keys = *items* and values = *their locations in the linked list (i.e., iterators to the corresponding nodes)*. When an item is accessed or removed, update the linked list and the symbol table accordingly:

- When you **access** an element, remove it from its current position (if it exists) and re-insert it at the beginning of the list. Update its entry in the symbol table to point to the new position.
- When you **remove** an element, delete it from the end of the list and remove its entry from the symbol table.
- After an **access**, if the cache holds more than *capacity* items, call **remove** to evict one.

Implement a class LRU with the following methods:

Function	Description
LRU(int capacity)	Create an empty cache that holds at most capacity items.
access(int item)	Insert item if not already present, evicting the least recently accessed item if the cache is over capacity; mark item as the most recently accessed.
remove()	Remove and return the least recently accessed item; throw std::out_of_range if the cache is empty.
print() const	Print the items in order of most recent to least recent access.
contains(int item) const	Return true if item is present.
size() const	Return the number of items in the cache.
empty() const	Return true if the cache is empty.

Following is a sample usage of the LRU class:

```

1 #include "LRU.h" // test-lru.cpp
2 int main() {
3     LRU cache(3); // holds at most 3 items
4     cache.access(10); cache.access(20); cache.access(30);
5     std::cout << "Cache after 10,20,30: "; cache.print(); // Expected: 30 20 10
6
7     cache.access(20); // move 20 to front
8     std::cout << "After accessing 20: "; cache.print(); // Expected: 20 30 10
9
10    cache.access(40); // cache is full: evicts 10
11    std::cout << "After accessing 40: "; cache.print(); // Expected: 40 20 30
12    std::cout << "Contains 10? " << cache.contains(10) << "\n"; // Expected: 0 (false)
13
14    std::cout << "Remove LRU = " << cache.remove() << "\n"; // Expected: 30
15    std::cout << "Cache after remove: "; cache.print(); // Expected: 40 20
16
17    std::cout << "Size = " << cache.size() << "\n"; // Expected: Size = 2
18    std::cout << "Empty = " << cache.empty() << "\n"; // Expected: Empty = 0 (false)
19 }
```

Part C: Mutable string

Create a data type `Mstring` that supports the following operations on a string in $O(\log n)$ time in the worst case, where n is the length of the string:

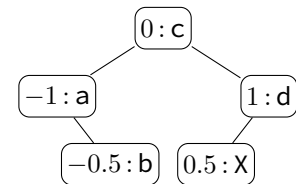
Function	Description
<code>get(int i)</code>	Return the i -th character of the string.
<code>insert(int i, char c)</code>	Insert <code>c</code> so that it becomes the i -th character of the string.
<code>remove(int i)</code>	Delete the i -th character from the string.
<code>size()</code>	Return the length of the string.
<code>print()</code>	Print the entire string.

Hint: Represent the string using the AVL tree in `avl.hpp` with `key` = a real number (`double`) and `value` = a character, i.e., `AVL<double, char>`. It has the same interface as the BST from the lectures (`get`, `put`, `select`, `rank`, `remove`, iterators, ...), and since an AVL tree has height $O(\log n)$, each of these takes $O(\log n)$ time. The characters of the string are the values of the tree in inorder, so the i -th character is the value whose key has rank i .

- `get(i)`: use `select(i)` to find the key of rank i , then look up its value.
- `insert(i, c)`: give `c` a key that lies between the keys of ranks $i - 1$ and i — their average. To insert at the front, use the smallest key minus 1; to insert at the end, use the largest key plus 1; in an empty string, use 0.
- `remove(i)`: remove the key of rank i from the tree.

Example. The table shows the key given to each character by a sequence of `insert` calls, and the figure shows the resulting AVL tree (each node is `key:character`). Reading the tree in inorder gives the string `abcXd`; for example, `get(3)` calls `select(3)`, which returns the key 0.5, whose value is `X`.

Call	Key of the new character	Rule	String
<code>insert(0, 'c')</code>	0	empty	c
<code>insert(1, 'd')</code>	$0 + 1 = 1$	end	cd
<code>insert(0, 'a')</code>	$0 - 1 = -1$	front	acd
<code>insert(1, 'b')</code>	$(-1 + 0)/2 = -0.5$	middle	abcd
<code>insert(3, 'X')</code>	$(0 + 1)/2 = 0.5$	middle	abcXd



Note: each average halves the gap between two keys, so about 50 insertions at the *same* position exhaust the precision of a `double`. You do not need to handle this.

Following is a sample usage of the `Mstring` class:

```

1 #include "Mstring.h" // test-mstring.cpp
2 int main() {
3     Mstring s;
4     // Insert characters to form "abcd"
5     s.insert(0, 'a'); s.insert(1, 'b'); s.insert(2, 'c'); s.insert(3, 'd');
6     std::cout << "String after inserts: "; s.print(); // Expected: abcd
7     // Get a character
8     std::cout << "s.get(2) = " << s.get(2) << std::endl; // Expected: c
9     // Insert in the middle
10    s.insert(2, 'X');
11    std::cout << "After insert X at pos 2: "; s.print(); // Expected: abXcd
12    // Remove a character
13    s.remove(3);
14    std::cout << "After remove at pos 3: "; s.print(); // Expected: abXd
15    // Remove first and last
16    s.remove(0); s.remove(s.size() - 1);
17    std::cout << "After removing first and last: "; s.print(); // Expected: bX
18 }
  
```